Class Activity
File System Analyzer

> ⚠️ **GenAI Level 1 — Limited AI Use**
>
> This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the **Generative AI Use in Assessments** policy posted at the top of the **Assignments** section in Moodle.

Objective

Create a C# console application that explores a folder on the computer and produces a useful summary of its contents. The activity combines **file and directory handling**, **operators/flow control**, and **strings/data types**. These topics align with the supplied course material on C# types and strings and operators, conditions, and flow control.

Do not hard-code the results. Your program must actually inspect the file system.

Scenario

You are developing a small **File System Analyzer** for an IT department. The program must allow a user to enter a folder path and then analyze the files and subdirectories contained in that location.

For example, a user might analyze:

`C:\Users\Student\Documents`

or another valid folder on the computer.

Requirements

Your application must first ask the user to enter a directory path. It must determine whether the directory exists. If it does not exist, display an appropriate error message rather than allowing the program to crash.

For a valid directory, display the directory's **name, full path, and creation date**.

Then analyze its **immediate subdirectories**. For every subdirectory, display its name and creation date. Also display the **total number of subdirectories** found.

Next, analyze the **files** contained directly in the selected directory. For every file, display its name, extension, size in bytes, and last modified date.

Your program must then classify each file according to its size:

File Size	Classification
Less than 10 KB	Small
10 KB to less than 100 KB	Medium
100 KB or larger	Large

Use appropriate C# operators and flow-control structures to determine the classification.

File-Type Analysis

Using each file's extension, classify it into one of these categories: **Text/Document**, **Image**, **Code**, or **Other**.

At minimum, your application must recognize **`.txt`**, **`.pdf`**, **`.jpg`**, **`.png`**, **`.cs`**, and **`.java`**.

The comparison should work even if an extension contains uppercase letters—for example, .JPG should be treated the same as **`.jpg`**. This requires appropriate string processing rather than writing duplicate conditions for every possible capitalization.

Summary

After processing the directory, display a summary containing the **number of files, number of subdirectories, total size of all files, number of Small/Medium/Large files, and number of files in each file-type category**.

Also identify and display the **largest file** in the directory, including its name and size.

Challenge Requirement

Search through the immediate subdirectories as well. For each subdirectory, determine how many files it contains and display:

`Subdirectory Name → Number of Files`

Then determine which immediate subdirectory contains the **largest number of files**.

You must decide which loops, conditions, variables, and file/directory classes are appropriate. Recall that foreach is particularly suited to processing every element of a collection without manually managing an index.

Expected Console Structure

Your output should be organized approximately as follows, but you may design your own formatting:

```
===== FILE SYSTEM ANALYZER =====
```

```
Directory:
Name:
Full Path:
Created:

----- SUBDIRECTORIES -----

...

----- FILES -----

File:
Extension:
Size:
Modified:
Category:
Size Classification:

...

----- ANALYSIS SUMMARY -----

Files:
Subdirectories:
Total Size:
Small Files:
Medium Files:
Large Files:

Text/Documents:
Images:
Code Files:
Other:

Largest File:
Largest File Size:

Subdirectory With Most Files:
```

Submission

Show the **completed working application to the instructor during class**. Be prepared to run it using at least **two different directory paths** and briefly explain how your program:

- obtains files and directories,
- processes them using loops,

- classifies files using conditions,
- calculates the summary,
- determines the largest file and the subdirectory containing the most files.

No starter code is provided. You are expected to design and implement the solution yourself.